

FACULTAD DE INGENIERÍA
FUNDAMENTOS DE COMPUTACION Y PROGRAMACION

PRUEBA PARCIAL PROGRAMADA N°2



ASPECTOS GENERALES DE LA PRUEBA

- Lea atentamente la prueba y las instrucciones antes de comenzar a desarrollarla.
- Queda prohibido hablar con los compañeros(as) durante el desarrollo de la PEP.
- La PEP contiene 2 preguntas de desarrollo, con un total de **45** puntos y una exigencia del **60%**
- Tiene un límite de tiempo de **90** minutos para responder.
- **El equipo docente tiene la prohibición de responder consultas.**
- El/La estudiante que se sorprenda en actos deshonestos será calificado con la nota mínima.
- Los elementos tecnológicos deben permanecer apagados y guardados. Queda absolutamente prohibido el uso todo elemento tecnológico. Su uso puede significar la nota mínima o sanciones mayores.
- El alumno deberá identificarse con su Cédula de Identidad.
- Sobre el escritorio sólo podrá existir lápiz (obligatorio) y goma/lápiz corrector (opcional).
- Complete sus datos personales antes de comenzar la evaluación.
- Considere que la evaluación contempla el código, los comentarios y el seguimiento de las buenas prácticas de programación.
- Responda cada pregunta, a continuación de su enunciado, en el espacio que se le entrega para ello.

NOMBRE	RUT	SECCIÓN	

1. (30 puntos) CONVERSIÓN DE ARCHIVOS

Los computadores, en la actualidad, almacenan su información de manera permanente en un componente de hardware llamado disco duro, este almacena la información en Bytes.

Un Byte es una unidad de información que comúnmente equivale a un conjunto de 8 bits, es decir, una combinación de 8 valores distintos de 0's y 1's. Sin embargo, como toda unidad de medición, existen abreviaturas para evitar la escritura números grandes, los que para el caso de los Bytes (En sistemas Windows) son:

- Kilobyte (KB): 2^{10} Bytes
- Megabyte (MB): 2^{20} Bytes
- Gigabyte (GB): 2^{30} Bytes
- Terabyte (TB): 2^{40} Bytes
- Petabyte (PB): 2^{50} Bytes

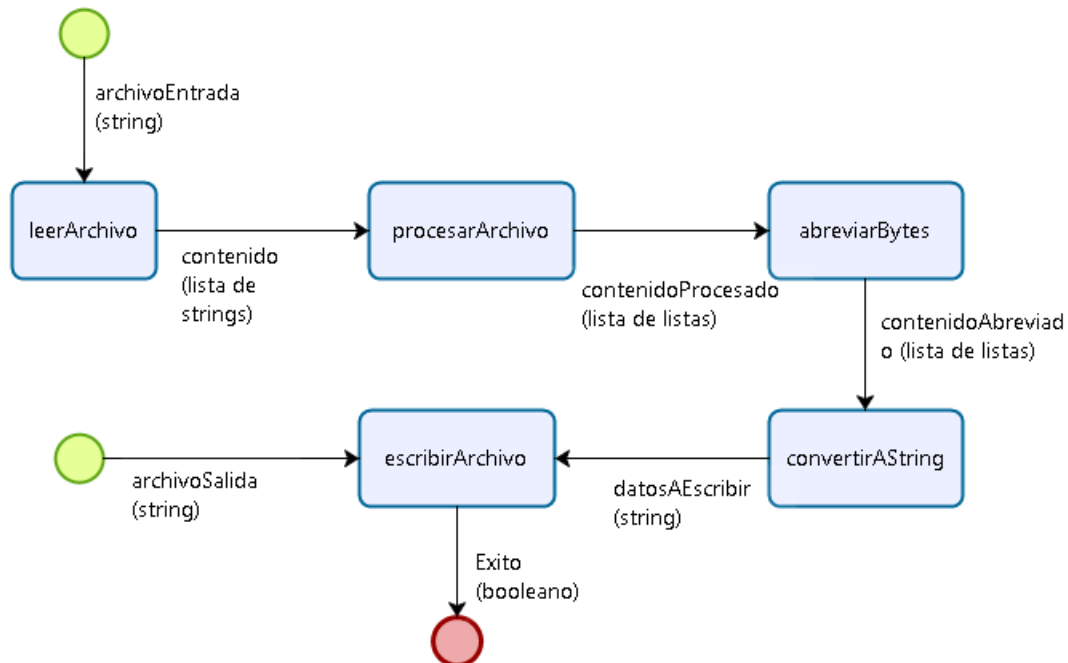
Para un usuario, es mucho más fácil dimensionar los tamaños de archivos si estos se representan de forma aproximada, sin embargo, los computadores al sólo entender Bytes, nos entregan la información de esta manera.

A fin de hacer más fácil para un usuario el dimensionar el tamaño de un archivo, se requiere construir un programa que reciba un listado de archivos "*directorio.txt*" dónde cada línea es un par nombre de archivo (con extensión) y el espacio que ocupa (en bytes) y construya una salida "*directorioAbreviados.txt*" con los tamaños abreviados con dos decimales de precisión. Un ejemplo del proceso con la entrada y salida esperada, puede verse a continuación:

directorio.txt Curriculum Vitae.docx, 40246 Game.of.Trones.S01E03720p, 2245646567 Queen-02_We_Are_The_Champions, 45752567 tareaQueMandoElProfeYNoTermine.py, 15
--

directorioAbreviados.txt Curriculum Vitae.docx, 39.30 KB Game.of.Trones.S01E03720p, 2.09 GB Queen-02_We_Are_The_Champions, 43.63 MB tareaQueMandoElProfeYNoTermine.py, 15.00 B

A fin de facilitar la solución se le ha entregado la siguiente estrategia de solución:



El que puede leerse como:

- La función `leerArchivo()` recibe como entrada el nombre del archivo a leer y entrega como salida una lista de strings dónde cada elemento de la lista es una línea del archivo (con salto de línea incluido), por ejemplo:
'Curriculum Vitae.docx; 40246\n'
- La función `procesarArchivo()` recibe el contenido y los ordena como una lista de listas dónde cada elemento será un par [string, int] de la forma:
['Curriculum Vitae.docx', 40246]
- A partir de dicha salida la función `abreviarBytes()` aplica el proceso de abreviatura a cada tamaño de archivo, y lo deja en una listas de listas de la forma:
['Curriculum Vitae.docx', '43.63 MB']

- La función `convertirAString()`, transforma el contenido a un string para asegurar que el proceso siguiente realice la escritura.
- Finalmente la función `escribirArchivo()`, recibe como entrada los datos a escribir y el nombre del archivo, para construir el archivo de salida y entrega un `True`, para que el programa informe cuando el proceso termina exitosamente.

A partir de este diseño de solución, se le solicita a usted que, a partir de la abstracción generada, construya:

- (9 puntos)** Las funciones `convertirAString()` y `escribirArchivo()`,
- (15 puntos)** La función `abreviarBytes()`,
- (6 puntos)** El bloque principal del programa

Considere que:

- Las funciones `leerArchivo()` y `procesarArchivo()` existen y funcionan correctamente.
- Las listas anteriores representan ejemplos y que el programa debe funcionar para todos los casos que respondan a las reglas anteriores.
- La función nativa de Python `round()`, recibe como entrada un valor (`int`, `long` o `float`), una cantidad de decimales (`int`) y entrega la aproximación del número en flotante con la cantidad de decimales solicitados.
- Es posible definir funciones auxiliares siempre y cuando éstas no alteren la abstracción dada.

SOLUCIÓN

ITEM A

```
# Función que convierte una lista de listas a una lista de strings
# ENTRADA: lista de pares [nombre archivo (string), bytes abreviados (string)
# SALIDA: Texto
def convertirAString(lista):
    # Se declara un iterador
    i = 0
    # Se declara un string vacío para almacenar el texto
    texto = ''
    # Mientras el iterador no alcance el largo de la lista
    while i < len (lista) :
        # Convierte el contenido de la sublista a un único string con salto de
        línea
        linea = lista[i][0] + ', ' + lista[i][1] + '\n'
        # Se agrega la línea al texto
        texto = texto + linea
        # Se incrementa el iterador
        i = i + 1
    # Se devuelve el texto
    return texto
```

```

# Función que escribe un string en un archivo de texto
# ENTRADA: datos a escribir (string), nombre de archivo (string)
# SALIDA: Booleano True si el proceso se realizó correctamente
def escribirArchivo(datosAEscribir, nombreArchivo):
    # Se abre el archivo en modo de escritura
    archivo = open(nombreArchivo, 'w')
    # Se escriben los datos en el archivo de salida
    archivo.write(datosAEscribir)
    # Se cierra el archivo
    archivo.close()
    # Se retorna un True
    return True

```

ITEM B

```

# Función que realiza la abreviatura de los bytes
# ENTRADA: lista de pares [nombre archivo (string), bytes (int)]
# SALIDA: lista de pares [nombre archivo (string), bytes abreviados (string) ]
def abreviarBytes(contenido):
    # Se declara un iterador para recorrer la lista
    i = 0
    # Mientras i no alcance el largo del contenido
    while i < len(contenido):
        # Se revisa si la cantidad de bytes cae en la categoría de PetaBytes
        if contenido[i][1] > 2 ** 50 :
            # Se divide el valor en Bytes por el valor de la potencia
            valor = contenido[i][1]/(2**50.0)
            # Se redondea el valor con dos decimales,
            # Se convierte a string y
            # Se le agrega el sufijo
            valor = str(round(valor,2)) + ' PB'
        # Se revisa si la cantidad de bytes cae en la categoría de TeraBytes
        elif contenido[i][1] > 2 ** 40 :
            # Se divide el valor en Bytes por el valor de la potencia
            valor = contenido[i][1]/(2**40.0)
            # Se redondea el valor con dos decimales,
            # Se convierte a string y
            # Se le agrega el sufijo
            valor = str(round(valor,2)) + ' TB'
        # Se revisa si la cantidad de bytes cae en la categoría de GigaBytes
        elif contenido[i][1] > 2 ** 30 :
            # Se divide el valor en Bytes por el valor de la potencia
            valor = contenido[i][1]/(2**30.0)
            # Se redondea el valor con dos decimales,
            # Se convierte a string y
            # Se le agrega el sufijo
            valor = str(round(valor,2)) + ' GB'
        # Se revisa si la cantidad de bytes cae en la categoría de MegaBytes
        elif contenido[i][1] > 2 ** 20 :

```

```

        # Se divide el valor en Bytes por el valor de la potencia
        valor = contenido[i][1]/(2**20.0)
        # Se redondea el valor con dos decimales,
        # Se convierte a string y
        # Se le agrega el sufijo
        valor = str(round(valor,2)) + ' MB'
    # Se revisa si la cantidad de bytes cae en la categoría de KiloBytes
    elif contenido[i][1] > 2 ** 10 :
        # Se divide el valor en Bytes por el valor de la potencia
        valor = contenido[i][1]/(2**10.0)
        # Se redondea el valor con dos decimales,
        # Se convierte a string y
        # Se le agrega el sufijo
        valor = str(round(valor,2)) + ' KB'
    # Si no cae en ninguna categoría anterior, entonces son Bytes
    else :
        # Se divide el valor en Bytes por el valor de la potencia
        valor = contenido[i][1]/(2**0.0)
        # Se redondea el valor con dos decimales,
        # Se convierte a string y
        # Se le agrega el sufijo
        valor = str(round(valor,2)) + ' B'
    # Se actualiza el valor en la lista de contenidos
    contenido[i][1] = valor
    # Se aumenta el iterador para revisar el siguiente elemento
    i = i + 1
# Se retorna el contenido
return contenido

```

ITEM C

```

# BLOQUE PRINCIPAL

# ENTRADA
# Se invoca la función para leer el archivo
contenido = leerArchivo("directorio.txt")
# PROCESAMIENTO
# Se convierte el contenido de archivo en una lista de listas
contenido = procesarArchivo(contenido)
# Se realiza el proceso de abreviatura de Bytes
contenido = abreviarBytes(contenido)
# Se construye el string de salida
salida = convertirAString(contenido)

# SALIDA
# Si, se escribe la salida correctamente
if escribirArchivo(salida, "directorioAbreviado.txt"):
    # Se informa al usuario que el proceso terminó correctamente
    print "El proceso fue realizado exitosamente"

```

2. (15 puntos) APROXIMACIÓN DE TAYLOR

Una serie de Taylor es una aproximación de funciones mediante una serie de potencias o suma de potencia enteras de polinomios. Por ejemplo, la serie a continuación, aproxima e^x :

$$e^x \approx \sum_{i=0}^n \frac{x^i}{i!}$$

Deseamos saber para un valor de n dado, que tan buena es la aproximación de la serie de Taylor para un vector x .

Para ello, construya un programa en Python que, a partir del valor inicial del vector x , el valor final del vector x y n , las iteraciones deseadas, grafique la aproximación por serie de Taylor en azul, y el valor real de e^x en rojo.

Para su solución considere que:

- Todo gráfico debe estar correctamente rotulado y titulado.
- Ya se ha definido una función `factorial()` que recibe como entrada un número y entrega como resultado el factorial de ese número.
- Numpy permite importar el valor de e .

```
# BLOQUE DE DEFINICIÓN
# IMPORTACIÓN DE FUNCIONES Y CONSTANTES
# Se importa numpy
import numpy
# Se importa matplotlib con el alias plotter
import matplotlib.pyplot as plotter

# BLOQUE PRINCIPAL
# ENTRADA
# Se solicita el valor inicial de x
xIni = input("Ingrese el valor inicial de x: ")
# Se solicita el valor final de x
xFin = input("Ingrese el valor Final de x: ")

# Se solicita el número de iteraciones
n = input("Indique el valor de n: ")
# PROCESAMIENTO

# Se crea un vector X con los valores iniciales y finales de x,
# con 1000 puntos para graficar
vectorX = numpy.linspace(xIni, xFin, 1000)
# Se crea un vector e utilizando el valor de e de numpy y el vector X
vectorE = numpy.e**vectorX
# Se inicializa un vector con ceros para almacenar los valores de
# las aproximaciones de la serie de Taylor
taylor = numpy.zeros(len(vectorX))
```

```

# Se declara un iterador para realizar las iteraciones necesarias
i = 0
# Mientras i no alcance el valor de n
while i <= n:
    # Se eleva el vector X por i (escalar)
    # se divide por el factorial de i (escalar)
    # Se almacena el valor parcial en el vector resultado
    resultado = vectorX**i / factorial(i)
    # Se agrega el resultado al vector Taylor
    # que actua como acumulador
    taylor = taylor + resultado
    # Se incrementa el valor de i
    i += 1

    # SALIDA
# Se genera la curva de e ** x
curvaEX = plotter.plot(vectorX, vectorE , label="e**x")
# Se genera la curva de la aproximación de la serie de Taylor
curvaTaylor = plotter.plot(vectorX, taylor, label="Taylor")
# Se le asigna el color indicado a la aproximación
plotter.setp(curvaTaylor,'color', 'b')
# Se le asigna el color indicado al valor de e
plotter.setp(curvaEX,'color', 'r')
# Se titula el gráfico
plotter.title("Comparacion aproximacion de taylor")
# Se etiquetan los ejes x e y
plotter.xlabel("x")
plotter.ylabel("e**x")
# Se muestran las leyendas para identificar las curvas
plotter.legend()
# Se muestra el gráfico
plotter.show()

```